26-1 이니로
알고리즘 멘토링

멘토 - 김수성

멘토링 진행 방식

기본 과제 풀이 C++

알고리즘 설명

문제 설명

X요일 20 ~ 21시

과제

추천 문제(G5 ~ P3)

필수는 아님

사용하고 싶은 언어 사용 (Python, C++ …)

과제

추천 문제 목록

새로 만들기

정렬 2개 + 필터

Aa 이름	문제 티어	주차	링크	멘토 풀이	성현님	지후님
랜선 자르기	S2	1주차	acmicpc.net/pro...m/1654	acmicpc.net/sou...8f9c92	☐	☐
휴게소 세우기	G4	1주차	acmicpc.net/pro...m/1477	acmicpc.net/sou...749e95	☐	☐
공유기 설치	G4	1주차	acmicpc.net/pro...m/2110	acmicpc.net/sou...4d1496	☐	☐
공장 컨설턴트 호석	G3	1주차	acmicpc.net/pro.../22254	acmicpc.net/sou...f5ecac	☐	☐
경비행기	G2	1주차	acmicpc.net/pro...m/2585	acmicpc.net/sou...5e5dc5	☐	☐
K번째 수	G1	1주차	acmicpc.net/pro...m/1300	acmicpc.net/sou...519821	☐	☐
Extraterrestrial Creatures	G1	1주차	acmicpc.net/pro.../34866	acmicpc.net/sou...09e86a	☐	☐
K Best	P5	1주차	acmicpc.net/pro...m/3639	acmicpc.net/sou...00d682	☐	☐
수열과 개구리	G2	번외	acmicpc.net/pro.../32294	acmicpc.net/sou...09d2a2	☐	☐
소가 길을 건너간 이유 7	G2	번외	acmicpc.net/pro.../14461	acmicpc.net/sou...53a5b4	☐	☐
그래서 나는 사진을 그만두었다	G2	번외	acmicpc.net/pro.../30094	acmicpc.net/sou...e687a6	☐	☐
트리의 높이와 너비	G2	번외	acmicpc.net/pro...m/2250	acmicpc.net/sou...7c188f	☐	☐
배열 정렬	G1	번외	acmicpc.net/pro.../28707	acmicpc.net/sou...ba8827	☐	☐
도로 검문	G1	번외	acmicpc.net/pro...m/2307	acmicpc.net/sou...b102d9	☐	☐
구두 수선공	G1	번외	acmicpc.net/pro.../14908	acmicpc.net/sou...15fa42	☐	☐
貰きピラミッド (Pyramid)	P5	번외	acmicpc.net/pro.../24129	acmicpc.net/sou...3f7ded	☐	☐

주차 별 내용

1주차	3/19	Parametric Search	5주차	5/7	Trie
2주차	3/26	Union Find	6주차	5/14	DP1
3주차	4/2	MST	7주차	5/21	DP2
4주차	4/9	Sparse Table	8주차	5/28	Segment Tree

풀다가 막힐 때..

디코, 카톡으로 질문하기 (과제가 아니여도 괜찮음)

풀이 검색해보고 문제 이해하기 or LLM 사용

더 좋은 강의 듣고 복습
kks227 블로그
https://blog.naver.com/kks227/220769859177

justicehui 블로그
https://justicehui.github.io/

질문?

1주차 – 매개 변수 탐색

매개 변수 탐색

매개 변수 탐색

최적화 문제를 결정 문제로 바꾸어
이분 탐색으로 문제를 해결 하는 것

결정 문제

Yes/No 로 답할 수 있는 문제

최적화 문제

최대값, 최소값을 찾는 문제

매개 변수 탐색

결정 문제

Yes/No 로 답할 수 있는 문제

최적화 문제

최대값, 최소값을 찾는 문제

최적화 문제를 해결 할 수 있으면
결정 문제를 항상 해결 할 수 있음

랜선 자르기 / 1654

백준 1654 / https://www.acmicpc.net/problem/1654

N개의 랜선을 잘라 같은 길이의 랜선 M개 이상을 만들어야 한다
랜선은 이어 붙일 수 없고, 자르고 남는 자투리는 버린다

모든 랜선은 정수 길이로만 자를 수 있을 때
만들 수 있는 랜선 길이의 최댓값을 구해라

랜선 자르기 / 1654

예제 입력 1 복사

```
4 11
802
743
457
539
```

예제 출력 1 복사

```
200
```

힌트

802cm 랜선에서 4개, 743cm 랜선에서 3개, 457cm 랜선에서 2개, 539cm 랜선에서 2개를 잘라내 모두 11개를 만들 수 있다.

랜선 자르기 / 1654

최적화 문제

최대값, 최소값을 찾는 문제

M개의 랜선을 만들 수 있는 랜선의 최대 길이

결정 문제

Yes/No 로 답할 수 있는 문제

랜선의 길이를 K로 잘랐을 때
M개 이상의 랜선을 만들 수 있는가

랜선 자르기 / 1654

결정 문제

Decision(K) = 랜선의 길이를 K로 잘랐을 때
M개 이상의 랜선을 만들 수 있는가

K = 0, 1, 2 … 에 대해서 결정 문제를 해결
Decision(K) = True 인 K의 최댓값을 찾으면 됨

모든 K에 대해서 문제를 해결해야 함 -> 비효율적

랜선 자르기 / 1654

입력	출력
4 11	200
802	
743	
457	
539	

Decision(K) = 랜선의 길이를 K로 잘랐을 때
M개의 이상의 랜선을 만들 수 있는가

Decision(K)가 1인 K의 최댓값이 정답
결정 문제의 값은 항상 정렬 되어 있음 -> 이분 탐색

위의 결정 함수에서 이상의 조건을 빼면
단조성(정렬)이 성립하지 않아서 이분 탐색 불가능

11 11 10 10

Decision(K)	…	1	1	1	1	0	0	0	0	…
K		197	198	199	200	201	202	203	204	

매개 변수 탐색

```
int maximization(){
    // 정답의 범위는 1 ~ N
    long long lo = 1, hi = N;
    while(lo < hi){
        long long mid = (lo + hi + 1) / 2; // 중간값
        // 결정 문제의 답이 1 이면
        // 정답은 [mid, hi]에 존재
        if(decision(mid)) lo = mid;

        // 결정 문제의 답이 0 이면
        // 정답은 [lo, mid - 1]에 존재
        else hi = mid - 1;
    }

    return lo;
}
```

```
int minimization(){
    // 정답의 범위는 1 ~ N
    int lo = 1, hi = N;
    while(lo < hi){
        int mid = (lo + hi) / 2; // 중간값
        // 결정 문제의 답이 1 이면
        // 정답은 [lo, mid]에 존재
        if(decision(mid)) hi = mid;

        // 결정 문제의 답이 0 이면
        // 정답은 [mid + 1, hi]에 존재
        else lo = mid + 1;
    }

    return lo;
}
```

Decision(K) 111111000000

000000111111

매개 변수 탐색

mid 값을 올림 하는 이유

lo와 hi의 차이가 1 일 때 올림을 하지 않으면

EX) lo = 5, hi = 6, mid = (5 + 6) / 2 = 5

decision(mid)가 1이면 lo = mid

항상 lo = 5, hi = 6, mid = 5로 갱신 됨

올림을 하면 mid = lo = 6으로 갱신되어서 무한루프 X

반대로 최솟값을 구할 때는 mid 값을 내림 해야함

mid = (lo + hi) / 2

랜선 자르기 / 1654

```
int maximazation(){
    long long lo = 1, hi = (1 << 31) - 1;
    while(lo < hi){
        long long mid = (lo + hi + 1) / 2;
        if(decision(mid)) lo = mid;
        else hi = mid - 1;
    }

    return lo;
}
```

```
bool decision(long long cur){
    long long cnt = 0;
    for(int i = 1;i <= n;i++){
        cnt += a[i] / cur;
    }

    return cnt >= m;
}
```

랜선 자르기 / 1654

시간 복잡도

결정 함수 -> O(N)

매개 변수 탐색 -> O(log 2^31) = 31

O(N * log (2 ^ 31))

질문?

공유기 설치 / 2110

백준 2110 / https://www.acmicpc.net/problem/2110

수직선 위의 N개 집 중 M개를 골라 공유기를 설치해야 한다.
한 집에는 공유기를 하나만 설치할 수 있다.

공유기를 설치한 뒤, 가장 가까운 두 공유기 사이 거리의 값을 최대화 해야 한다

즉, 공유기 M개를 배치할 때 가능한 최소 간격의 최댓값을 구해라

공유기 설치 / 2110

출력

첫째 줄에 가장 인접한 두 공유기 사이의 최대 거리를 출력한다.

예제 입력 1 복사

```
5 3
1
2
8
4
9
```

예제 출력 1 복사

```
3
```

힌트

공유기를 1, 4, 8 또는 1, 4, 9에 설치하면 가장 인접한 두 공유기 사이의 거리는 3이고, 이 거리보다 크게 공유기를 3개 설치할 수 없다.

공유기 설치 / 2110

예제 입력	예제 출력
5 3 1 2 8 4 9	3

1 4 9에 공유기를 설치하면 각 거리가 3 5가 됨
거리의 최솟값이 3보다 크게 공유기를 설치할 수 없음

공유기 설치 / 2110

최적화 문제

설치한 M개의 공유기 사이의 거리의 최솟값의 최댓값

결정 문제

공유기 사이의 거리의 최솟값이 K 일 때
M개 이상의 공유기를 설치 할 수 있는가

공유기 설치 / 2110

결정 문제

공유기 사이의 거리의 최솟값이 K 일 때
M개 이상의 공유기를 설치 할 수 있는가

최솟값을 크게 만들려면
공유기를 적게 설치해야 함

K 값이 커지면 공유기를 M개 이상 설치 할 수 없음

공유기 설치 / 2110

결정 문제

공유기 사이의 거리의 최솟값이 K 일 때
M개 이상의 공유기를 설치 할 수 있는가

K 값이 커지면 공유기를 M개 이상 설치 할 수 없음
-> decision(K) 1111100000
최댓값을 구하면 됨

공유기 설치 / 2110

결정 문제

공유기 사이의 거리의 최솟값이 K 일 때
M개 이상의 공유기를 설치 할 수 있는가

위의 결정 문제에서 이상의 조건을 빼면 단조성이 성립 X
-> decision(K) 11111000000 … 0011111

어느 순간 결정 문제의 정답이 다시 1로 바뀌는 순간이
생겨서 이분탐색이 불가능해짐

공유기 설치 / 2110

결정 문제

공유기 사이의 거리의 최솟값이 K 일 때
M개 이상의 공유기를 설치 할 수 있는가

처음에 공유기를 설치하고 전에 공유기를 설치한 장소와
거리가 K이상이 되었을 때 공유기를 설치함
-> 항상 공유기 사이의 거리는 K 이상

이 때 설치한 공유기 개수가 M개 이상이면
Decision(K)는 1이 됨

공유기 설치 / 2110

처음에 공유기를 설치하고 전에 공유기를 설치한 장소와
거리가 K이상이 되었을 때 공유기를 설치함

더 왼쪽에 둘 수록 이후 공유기를 놓을 공간이 줄지 않으므로
그리디하게 둘 수 있는 가장 왼쪽에 두는게 항상 이득

공유기 설치 / 2110

시간 복잡도

배열 정렬 -> O(N log N)

결정 함수 -> O(N)

매개 변수 탐색 -> O(log 1e9) = 30...

O(N log N + N log 1e9)

공유기 설치 / 2110

```
bool decision(ll cur){
    // cnt -> 설치한 공유기 개수
    // last -> 마지막으로 설치한 공유기 위치
    ll cnt = 0, last = -1e12; // last는 음의 무한대로 초기화
    for(int i = 1;i <= n;i++){
        // 마지막으로 설치한 공유기의 위치와
        // 현재 위치의 차이가 K보다 작다면 건너 뜀
        if(a[i] - last < cur) continue;
        last = a[i]; cnt++;
    }

    // 설치한 공유기 개수가 M 이상이면
    // 공유기 거리의 차이의 최솟값을 K로 만들 수 있음
    return cnt >= m;
}

ll maximization(){
    // 최솟값은 1, 최댓값은 1e9
    // 실수 오차 있을 수 있으니 넉넉하게 2e9로 잡음
    ll lo = 1, hi = 2e9;
    while(lo < hi){
        ll mid = (lo + hi + 1) / 2; // 올림 값 사용
        if(decision(mid)) lo = mid;
        else hi = mid - 1;
    }

    return lo;
}
```

질문?

K번째 수 / 1300

백준 1300 / https://www.acmicpc.net/problem/1300

A[i][j] = i × j인 N × N 배열을 만든다.
이 배열의 모든 원소를 1차원 배열 B에 넣은 뒤 오름차순 정렬한다

정렬된 배열 B에서 K번째에 있는 수를 구해라

K번째 수 / 1300

A[i][j] = i * j 일 때 N * N의 배열 A에서 K번째 수 출력

출력

B[k]를 출력한다.

예제 입력 1 복사

```
3
7
```

예제 출력 1 복사

```
6
```

K번째 수 / 1300

최적화 문제

배열 A에서 작거나 같은 수의 개수가 K개 이상인 수 중 최솟값

예제 입력	예제 출력
3	6
7	

1	2	3
2	4	6
3	6	9

6 보다 작거나 같은 수 -> 8개

4 보다 작거나 같은 수 - > 6개

4는 최적화 문제를 만족하지 못함

K번째 수 / 1300

최적화 문제

배열 A에서 작거나 같은 수의 개수가 K개 이상인 수 중 최솟값
중복인 수가 있을 수 있으므로 이상을 넣어야 함

결정 문제

현재 수가 M일 때
M보다 작거나 같은 수가 K개 이상인가

K번째 수 / 1300

결정 문제

현재 수가 M일 때
M보다 작거나 같은 수가 K개 이상인가

M이 커지면 M보다 작거나 같은 수가 늘어 남
decision(M) = 0000011111

최솟값을 구하면 됨

K번째 수 / 1300

결정 문제

현재 수가 M일 때
M보다 작거나 같은 수가 K개 이상인가

M보다 작거나 같은 수를 나이브하게 구하면
배열은 N*N(N <= 1e5) 이므로 무조건 시간 초과

빠르게 작거나 같은 수를 구하는 법은 없을까?

K번째 수 / 1300

결정 문제

현재 수가 M일 때
M보다 작거나 같은 수가 K개 이상인가

배열의 i행은 항상 i의 배수
각 행에 대해서 M를 i로 나누면
작거나 같은 수의 개수가 나옴

단 각 행의 원소 개수는 N이므로
최댓값은 N으로 제한해야 함

1x1	1x2	1x3
1x2	2x2	2x3
1x3	2x3	3x3

K번째 수 / 1300

```
bool decision(ll cur){
    // cur 보다 작은 수의 개수
    ll cnt = 0;
    for(int i = 1;i <= n;i++){
        // i번째 열의 cur 보다 작은 수의 개수를 더함
        cnt += min(cur / i, n);
    }

    return cnt >= m;
}

ll minimization(){
    // 범위의 최댓값은 1e10
    ll lo = 1, hi = 2e10;
    while(lo < hi){
        ll mid = (lo + hi) / 2;
        if(decision(mid)) hi = mid;
        else lo = mid + 1;
    }

    return lo;
}
```

K번째 수 / 1300

시간 복잡도

결정 함수 - O(N)

이분 탐색 – O(log 1e10 = 33…)

O(N log 1e10)

질문?

K Best / 3639

백준 3639 / https://www.acmicpc.net/problem/3639

N개의 보석 중에서 정확히 K개를 골라 남겨야 한다.
각 보석은 가치 v[i]와 무게 w[i]를 가지며,
선택한 보석들의 가치 합 / 무게 합을 최대화해야 한다

즉, 평균적인 가성비가 가장 좋은 k개의 보석을 고르는 문제이다.
정답으로는 조건을 만족하는 보석들의 번호를 아무거나 출력하면 된다.

K Best / 3639

N개의 보석 중에서 정확히 K개를 골라 남겨야 한다.
각 보석은 가치 v[i]와 무게 w[i]를 가지며,
선택한 보석들의 가치 합 / 무게 합을 최대화해야 한다

최적화 문제

K개의 보석의 가치 합 / 무게 합을 최대화

결정 문제

가치 합 / 무게 합의 값이 M이상 일 때
K개의 보석을 고를 수 있는가?

K Best / 3639

결정 문제

가치 합 / 무게 합의 값이 M이상 일 때

K개의 보석을 고를 수 있는가?

위의 결정 문제를 해결하기 위해서 식으로 바꿔보자

sum(v) / sum(w) >= M

sum(v) >= M * sum(w)

sum(v) – M * sum(w) >= 0

K Best / 3639

결정 문제

가치 합 / 무게 합의 값이 M일 때 K개의 보석을 고를 수 있는가?

sum(v) – M * sum(w) >= 0

위의 sum을 풀어보면 아래의 식을 얻을 수 있다

sum(v[0]) – M * sum(w[0]) + … +

sum(v[K – 1]) – M * sum(w[K – 1]) >= 0

K Best / 3639

결정 문제

가치 합 / 무게 합의 값이 M일 때 K개의 보석을 고를 수 있는가?

sum(v[0]) – M * sum(w[0]) + … +
sum(v[K – 1]) – M * sum(w[K – 1]) >= 0

결국 각 식은 i에 대해 종속적이고 M은 상수이므로
sum(v[i]) – M * sum(w[i])가
큰 순서대로 정렬 후 그리디하게 더 해주는 것이 최적이다

또한 가치 합 / 무게 합의 값이 실수이므로 실수 이분 탐색을 해야 함

K Best / 3639

실수 이분탐색은 정수와 다르게
올림/내림이 필요 없음

또한 실수 연산의 오차를 생각해서
적당히 작은 수인 eps 사용해서 보정

```
using ld = long double;
constexpr ld ebs = 1e-9;

struct node {
    int v, w, idx;
};
vector<node> arr;

bool f(ld cur) {
    sort(arr.begin(), arr.end(), [cur](node l, node r) {
        return (ld)l.v - cur * (ld)l.w > (ld)r.v - cur * (ld)r.w;
    });

    ld sum = 0;
    for (int i = 0; i < m; i++) sum += arr[i].v - cur * arr[i].w;
    if (sum < -ebs) return 0;

    result.clear();
    for (int i = 0; i < m; i++) result.push_back(arr[i].idx);
    return 1;
}

ld ret(ld l, ld r) {
    ld lo = l, hi = r, mid = -1;
    while (hi - lo > ebs) {
        mid = (hi + lo) / (ld)2;
        if (f(mid)) lo = mid;
        else hi = mid;
    }
    return lo;
}
```

K Best / 3639

시간 복잡도

결정 함수 - O(N log N)

이분 탐색 – O(log (1e7 / eps) = 54...)

O(N log N * log (1e7 / eps))

질문?

고생하셨습니다